CDot Language Documentation

v3.3.9 — core language + DLL/FFI and Native UI extensions

---

1. Introduction

CDot is a VM-based programming language with manual memory management and its own runtime.

The language is designed for:

- writing scripts
- building interactive programs and games
- developing custom DSLs (languages built on top of CDot)

> As of v3.3.9, the core language can be extended with two optional modules, enabled via `#include`: **dllapi.ddl** (calling external libraries through FFI) and **winapi.ddl** (native windows and UI controls). Extension commands are only available once the corresponding library is included — see Section 14.

2. Execution Model (VM)

The program executes line by line.

Core components:

- memory (object[])
- instruction pointer
- instruction set

Flow control:

- `jmp` — unconditional jump
- `jf` — conditional jump
- `call` / `ret` — functions
- `cmd` — command block declaration (bracket-free argument form, see §10)
- `map` — inline switch-case inside func/cmd (see §7)

3. Memory Model

All memory is:

```
object[] memory
```

Address = type + index. Types:

- `f` — float
- `s` — string
- `m` — system

Example:

```
0f  → number
1s  → string
```

Memory management: `made` and `not` operate directly on addresses:

Syntax	Description
`made addr`	MALLOC / DEALLOC — allocates memory at addr, or frees it if the address is already allocated (toggle behavior).
`not addr`	Logical/bitwise NOT applied to the value at addr; the result is written back to the same location.

4. System Ports

Syntax	Description
`0m`	output
`1m`	text color
`2m`	background
`3m`	sleep
`4m`	window title
`5m`	error
`6m`	CMD
`7m`	cursor
`8m`	resize console (may not work)
`9m`	show/hide console

5. Assignment

Syntax	Description
`mov dest source`	Assigns the value of source to dest.

6. Arithmetic & Bitwise

Basic arithmetic:

Syntax	Description
`add op1 op2 dest`	dest = op1 + op2
`sub op1 op2 dest`	dest = op1 - op2
`mul op1 op2 dest`	dest = op1 * op2
`div op1 op2 dest`	dest = op1 / op2

Syntax	Description
`xor op1 op2 dest`	Bitwise XOR: dest = op1 xor op2

Increment / decrement:

Syntax	Description
`inc reg target`	Increases the value at target by reg (or by 1 when used as a counter).
`dec reg target`	Decreases the value at target the same way as inc.

Executing an arbitrary command or evaluating a math expression:

Syntax	Description
`exc command/math res`	EXC — executes a CDot command, or evaluates a math expression passed as a string; when used for math, the result is stored in res.

7. Flow Control

Labels:

```
label:
jmp label
jf a > b label
```

`map` — inline switch-case branching on a value inside a `func`/`cmd`. The `pos` key is the default branch. The construct itself does not declare anything (it is not added to the function/command/label index):

```
map source
■case1: command
■pos: default_command
ret
```

8. Arrays

```
arr [1,2,3] nums
```

Commands:

Syntax	Description
`get array index dest_reg`	Reads the element of array at index into dest_reg.
`set array index source`	Writes source into array at index.
`len array dest_reg`	Writes the length of array into dest_reg.
`push array source`	Appends source to the end of array.
`pop array`	Removes the last element of array.

8.1 Stack (LIFO)

`push` / `pop` implement a stack.

8.2 Structures

```
arr [{a={b=10}}] data
```

Access:

```
get data 0.a.b 0f
```

8.3 Dynamic Indexing

The index can be a variable:

```
get nums 0f 1f
```

9. Strings

Supported escape sequences:

```
\n  \t  "
```

String operations:

Syntax	Description
`spt source separator target_array`	SPT — splits source by separator, storing the result in target_array.
`trm source dest_reg`	TRM — trims whitespace from both ends of source, result in dest_reg.
`low source dest_reg`	LOW — converts source to lowercase.
`big source dest_reg`	BIG — converts source to uppercase.

10. File System

Syntax	Description
`wrt file source`	Writes source to file.
`rdl file dest`	Reads the contents of file into dest.
`exs file result`	EXS — executes file (an external script/process); the result of execution is stored in result.

11. Input System

Syntax	Description
`key dest`	KEY — reads user input into dest.
`dlk`	DLK — asynchronous input (does not block execution).

12. Random / Utility

Syntax	Description
`rnd min max dest_reg`	Random number in the range [min, max].
`chr ascii_code dest_reg`	Character for the given ASCII code.

13. Screen

Syntax	Description
`cls`	Clears the screen.
`endl`	Line break.

14. Project Setup & Configuration

14.1 Creating a New Project

Running:

```
cdot new
```

scaffolds a new project in the current directory: it generates a `config.cfg` file with default settings and a `main.cdt` entry-point source file.

14.2 config.cfg Reference

config.cfg is an INI-style properties file (`Key = value`, `;` starts a comment) grouped into three sections.

Main settings

Syntax	Description
`MainStartFile = "main.cdt"`	The entry-point source file that gets executed when the project runs.
`AditionalContent = []`	A list of extra .ddl library filenames made available to the editor for autocomplete/indexing purposes. This does not by itself enable a library's commands in a .cdot/.cdt file — that still requires #include (see §14.3).
`Pause = false`	Whether the console/window stays open and waits for a key press after the program finishes, instead of closing immediately.
`Clear = false`	Whether the console is cleared right before the program starts running.

Window settings

Syntax	Description
`Title = ""`	The title shown in the console/window title bar.
`Icon = ""`	Path to an .ico file used as the application/window icon.

Language settings

Syntax	Description
`AllowCMD = true`	Whether the program is permitted to run shell commands through the CMD system port (6m).
`MaxMemory = 100`	The maximum number of cells available in the memory array (object[] memory) — an upper bound on addresses the program can allocate.
`Cursor = true`	Whether the console cursor is shown.
`Ending = true`	Whether an end-of-program indicator is shown once execution finishes (e.g. an exit/completion message).

Ending's exact behavior should be confirmed against the runtime you're targeting — it is documented here based on its config key name.

14.3 #include

#include — imports an external library (.ddl) and enables its associated command set:

Syntax	Description
`#include dllapi.ddl`	Enables the §17 commands: dll, @using, ffi.
`#include winapi.ddl`	Enables the native UI commands in §18 (winopen…) and the directives in §19 (@place…).

Libraries can also be referenced for editor indexing/autocomplete purposes via AditionalContent in config.cfg (§14.2) — this is separate from #include and does not enable extension commands in the source file itself.

Import order matters for std*.ddl libraries: a std*.ddl library (e.g. stdio.ddl) must be #include-d before any code in the file that calls its functions in dot notation (§15.2). Place these #include lines at the very top of the file, above everything else — including one further down, or after code that already calls libname.funcname, will not work.

15. Syntax Notes

15.1 Statement Separator (/)

A forward slash `/` lets you place several statements on a single line. It is purely a formatting shortcut — each part is executed as if it were written on its own line, in order, left to right:

```
@bg "form" "#000000" / @icon "form" "icon.ico"
mov 0m 2s / endl
```

This is commonly used to pair a value-producing statement with a follow-up action on the same line, e.g. printing a value and then immediately emitting a line break with endl.

15.2 Library Function Calls (dot notation)

Once a library is loaded via `#include`, some of its functions are called as `libname.funcname` instead of being exposed as bare global commands. Arguments follow the same positional rules as built-in commands:

```
stdio.pop dest
stdio.replace source "÷" "/"
```

> Which functions a given library exposes as bare commands (like winopen) versus dotted calls (like stdio.pop) depends on that library — check the library's own documentation for its full function list.
>
> Reminder: as noted in §14.3, std*.ddl libraries must be #include-d at the top of the file before their dot-notation functions are called anywhere below.

16. Functions & Call Stack

Function declaration (classic form, with bracketed argument list):

```
func [0f, 1s] myFunc
■add 0f 1f 2f
■mov 0m 2f
ret
```

Call:

```
call [5,10] myFunc
```

cmd — an alternative command-block declaration, without brackets around the arguments:

```
cmd myCmd arg1 arg2
■...
ret
```

16.1 Argument Passing

Arguments:

- passed by value
- can be literals or addresses

16.2 Call Stack

Each call:

- → pushes the current position

ret:

- → returns

16.3 Nested Calls

Functions can call other functions.

16.4 Errors

Errors are reported through system port 5m.

17. DLL / FFI Extension

Available only when `#include dllapi.ddl` is present.

Syntax	Description
`dll path`	Registers an external library at path, making it available for subsequent calls.
`@using path`	Sets path as the active DLL — subsequent ffi calls target it by default.
`ffi class method result args`	Calls method of class from the active (or explicitly given) library, passing args; the result is stored in result.

Example:

```
#include dllapi.ddl

dll "mathlib.dll"
@using "mathlib.dll"
ffi MathClass Square 0f 5
mov 0m 0f
hlt
```

18. Native UI Extension — Window & Controls

Available only when `#include winapi.ddl` is present.

17.1 Window

Syntax	Description
`winopen title width height`	Opens a native window with the given title and width×height dimensions.
`winclose`	Closes the current window.
`winwait result timeoutMs`	Blocks execution until a UI event occurs (e.g. a button click, key press, or key release) or timeoutMs elapses. The event is written to result — see below.

winwait return values: result normally holds the name of the triggering control (e.g. a button's name). For keyboard events it holds one of the following special formats instead:

Syntax	Description
`__keydown__:Key`	A key was pressed down; Key identifies which key.
`__keyup__:Key`	A key was released; Key identifies which key.
`__keychar__:char`	A character was typed; char is the resulting character.
`__closed__`	The window was closed (e.g. via the close button).

17.2 Controls

Syntax	Description
`winlbl "name" "text"`	Adds a text label.
`winbtn "name" "text"`	Adds a button; on click its name is pushed to winwait.

Syntax	Description
`wintxt "name" "text"`	Adds a single-line text input.
`wincheck "name" "text"`	Adds a checkbox.
`winradio "name" "text"`	Adds a radio button.
`wincombo "name" "item1|item2|item3"`	Adds a dropdown list.
`winlist "name" "item1|item2"`	Adds a scrollable list box.
`winprogress "name"`	Adds a progress bar.
`winslider "name"`	Adds a slider (trackbar).

17.3 Containers

Syntax	Description
`wingroup "name" "title"`	A titled group box container.
`winpanel "name"`	A plain panel container.
`winpbox "name"`	A picture box container for displaying an image.

17.4 Getters / Setters

Syntax	Description
`wingettext "name" resultVar`	Reads the text of control name into result.
`winsettext "name" value`	Sets the text of control name.
`wingetchecked "name" resultVar`	Reads the checked state of checkbox/radio name.
`wingetvalue "name" resultVar`	Reads the value of progress bar/slider name.

19. Native UI Extension — Style & Layout Directives

Directives address a control or window by name and configure its position, size, and appearance.

Syntax	Description
`@place "name" x y`	Absolute positioning.
`@grid "name" col row`	Positioning on a virtual grid.
`@scale "name" width height`	Sets width/height.
`@resizable addr/number`	Enables/disables window resizing.
`@getscale "name" widthResultVar heightResultVar`	Reads the current width/height into widthResult/heightResult.
`@bg "name" "#hex"`	Background color.
`@color "name" "#hex"`	Font color.
`@font "name" "fontFamily"`	Font family.

Syntax	Description
`@size "name" fontSize`	Font size.
`@visible "name" true`	Shows/hides a control.
`@enabled "name" true`	Enables/disables a control.
`@align "name" left`	Alignment: left / center / right.
`@border "name" none`	Border: none / single / fixed3d.
`@checked "name" true`	Checkbox/radio state.
`@icon "name" "path.ico"`	Window or control icon.
`@img "name" "path.img"`	Image for a control (e.g. winpbox).
`@value "name" number`	Progress bar/slider value.
`@range "name" min max`	Progress bar/slider value range.
`@additem "name" "item"`	Adds an item to a combo/list box.
`@clearitems "name"`	Clears a combo/list box.
`@parent "child" "container"`	Reparents child into container (group/panel).

20. Example

Basic example (core language only):

```
func [0f, 1f] sum
■add 0f 1f 2f
■mov 0m 2f
ret

call [3,4] sum
hlt
```

Native UI example (winapi.ddl):

```
#include winapi.ddl

winopen "Demo" 300 150
winbtn "btnOk" "OK"
@place "btnOk" 100 60

loop:
winwait 0s 0
jf 0s == "btnOk" onClick
jmp loop

onClick:
winclose
hlt
```

21. Notes

- memory is shared
- functions use registers
- the call stack drives execution
- DLL/FFI and Native UI commands are only enabled by an explicit #include of the corresponding .ddl library in the source file itself

---

END